Конспекты по Python

HexWolfCoder

17 октября 2026 г.

Содержание

1 Лекция 1. Инструментарий разработчика **2**
1.1 Что такое интерпретатор, REPL, байт-код , __pycache__ 2
1.1.1 Интерпретатор 2
1.1.2 REPL 4

1 Лекция 1. Инструментарий разработчика

Синопсис

Цель Настроить рабочее место, понять, что Python-код живёт в экосистеме.

Длительность 3–5 дней.

Теория (минимум)

- Что такое интерпретатор, REPL, байт-код, __pycache__.
- Версии Python: 3.10+, почему не 2.x.
- python, python3, pip, pipx, uv, poetry.
- venv: зачем изоляция зависимостей.
- pyproject.toml vs requirements.txt vs setup.py.
- Git, .gitignore для Python (.venv, `__pycache__`, `.pytest_cache`).
- Редактор: VS Code + Python extension или PyCharm.
- Terminal: bash/zsh, базовые командыю.

1.1 Что такое интерпретатор, REPL, байт-код , __pycache__

1.1.1 Интерпретатор

Python - интерпретируемый язык Это значит, что исходный код `.py` **не компилируется** в машинный код (python3) напрямую, а **выполняется** специальной программой — интерпретатором. Но это упрощение. На самом деле путь такой: (см. Рис.1). То есть Python сначала компилирует исходник в байт-код, а потом интерпретирует байт-код виртуальной машиной. Поэтому Python иногда называют **«компилируемо-интерпретируемым»** языком.

Что нужно понимать:

- **Интерпретатор** — это программа `python3`, которая читает `.py`, компилирует в байт-код и выполняет.
- **Интерпретатор** не создаёт `.exe`/бинарник. Чтобы запустить программу, нужен установленный Python.
- **Python-код** медленнее C, потому что каждая операция проходит через PVM.

`Интерпретатор` — Исходный код `.py` компилируется в байт-код.

Схема обработки и выполнения программы на Python

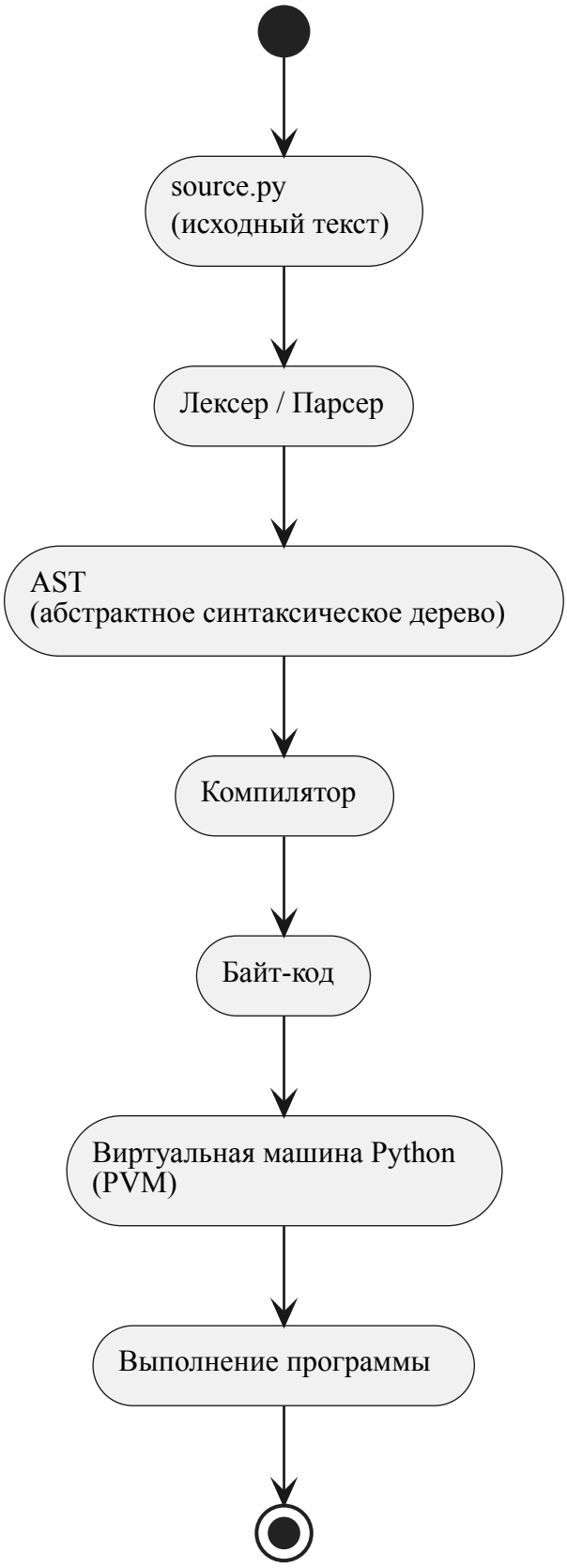


Рис. 1 – Схема обработки и выполнения программы на Python

1.1.2 REPL

REPL = Read-Eval-Print Loop. Это интерактивный режим интерпретатора.

```
$ python3
Python 3.12.0 (main, ...)
>>> 2 + 2
4
>>> name = "world"
>>> print(f"hello, {name}")
hello, world
>>> exit()
```

Цикл:

- Read — читает строку.
- Eval — вычисляет.
- Print — печатает результат.
- Loop — возвращается к началу.

Зачем нужен:

- Быстро проверить идею.
- Изучить поведение функции/метода.
- Отладка маленьких фрагментов.
- Прототипирование.

Улучшенные REPL:

- ipython — подсветка, автодополнение, ?, ??, %timeit.
- bpython, ptpython.
- Jupyter Notebook — REPL в браузере с сохранением состояния и визуализацией.

Что нужно понимать:

- **REPL** — не для продакшена, а для экспериментов.
- В **REPL** переменные живут до выхода.
- _ хранит последний результат.